

Resumen Software Pt.1

PARTE 1: Contenidos

- RUBY
- RAILS CRUD
- PROCESOS
- SCRUM
- HISTORIAS DE USUARIO
- ESTIMACIONES
- EJERCICIOS PRUEBA PRACTICA

1. RUBY:

- Metodo → Todo es un objeto
- Convenciones → snake_case →PascalCase
- lista = [1,2] → is_even? → true/false
- lista.reverse → !Cambia definitivamente
- irb → Consola de ruby
- Identacion 2 espacios
- Retorno Metodos:

```
def nivel
  return pokemon.nivel
  pokemon.nivel = 4
  pokemon.nivel
end
```

nivel → pokemon.nivel

```
def nivel
  variable = 5
```

```
end
```

```
nivel → retorna 5 xd
```

- Comparaciones / Strings / otros de clase

2.3 RAILS / CRUD

- CRUD → Create, Read, Update, Delete
- Controllers → Clase que controla interacciones con el Usuario, vista y el modelo
- Models → Permiten interactuar con la BASE DE DATOS
- Routes → Rutas para ser usadas por el usuario, se conecta con el controlador.

```
post "/pokemon", to: "pokemons#create")
```

`"/pokemons"` → endpoint // `pokemons` → controlador // `create` → metodo

- `db:migrate` → Se guardan migraciones: Modificar la BDD consistentemente e iterativamente
- `rails s` = `rails server` = `bin/rails server` → Inicializa el servidor
- `rails generate model Pokemon name:string`
 - `create_pokemons.rb`
- `rails generate migration CreatePokemon` → atributos opcionales (tiene que ser en singular)

```
create_table: pokemons do |t|  
  t.string :name  
  t.string :pokemon_type
```

- `rails console`

- `sudo service postgresql start` → Inicia postgres

4. Procesos

A) Cascada

- Cada etapa se ejecuta solo una vez (SECUENCIA LINEAL) y son etapas interdependientes.



Figure 2-2 Ilustración: Modelo en Cascada

Ventajas:

- **Simplicidad** conceptual
- **Claridad** en fases
- **Seguimiento**

Desventajas:

- **No Feedback**
- **Alto Riesgo** hasta etapas Avanzadas del proyecto
- **Cambios Difíciles**, es rígido

B) Iterativos

- Con cada **entrega**, esperamos el **feedback del usuario** para decidir los siguientes pasos a seguir.

- Aproximación **Gradual** al objetivo.
- **Desarrollo Modificable**

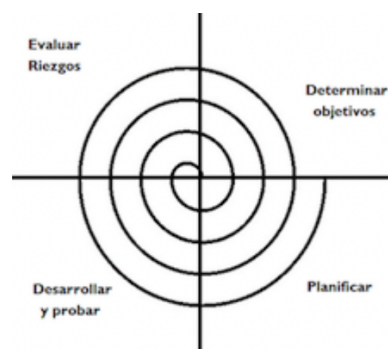


1. Modelo Espiral:

- Opera en ciclos y fases iterativas.
- Consideraciones de riesgo explícitas

4 Fases:

1. Análisis de Requisitos
2. Construcción
3. Evaluación
4. Análisis de riesgo



Ventajas:

- Manejo de **riesgo**
- **Flexibilidad**, se pueden presentar cambios

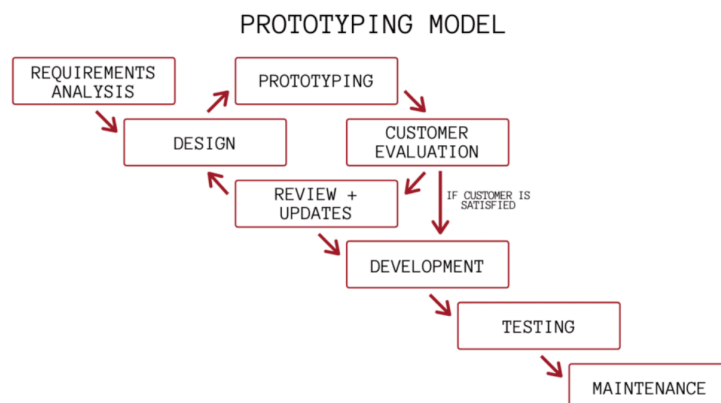
- Involucra **Retroalimentacion del Cliente**

Desventajas:

- Manejo del **Tiempo**: n° fases desconocido
- **Difícil** de estimar tiempo del proyecto
- **Inversion** importante de planeacion y analisis de riesgo

2. Modelo Prototipo:

- “Construccion de un producto que luce similar al objetivo principal”
 - Idea Central: **Usuario** tenga una **vista temprana** y **cercana al producto**
 - “El usuario solo sabra lo que queria cuando vea el resultado”
- 1. Prototipo Evolutivo: → El mismo prototipo evoluciona al **producto Final**
- 2. Prototipo Desechable: → Solo usado para vista rápida, se desecha y se empieza el producto **desde 0**



Ventajas:

- Reduce Improvistos → Menor tiempo de desarrollo

- Menor Frustracion y Ansiedad
- Satisfaccion del Usuario

Desventajas:

- Proyecto sin fin: Usuario **nunca Satisfecho**
- Expectativas de un "Casi listo" siempre
- Producto final no quedo como el prototipo

3. Modelo Unificado:

- Tiene su propio Lenguaje!!! → (UML)
- Contempla cuatro fases:
 1. Inicio: Establecimiento de casos, viabilidad
 2. Elaboración: Verificación de capacidad para construir el sistema
 3. Construcción: Edificacion de sistema que funcione (VERSION BETA)
 4. Transición: Entrega de sistema completamente funcional a Usuarios

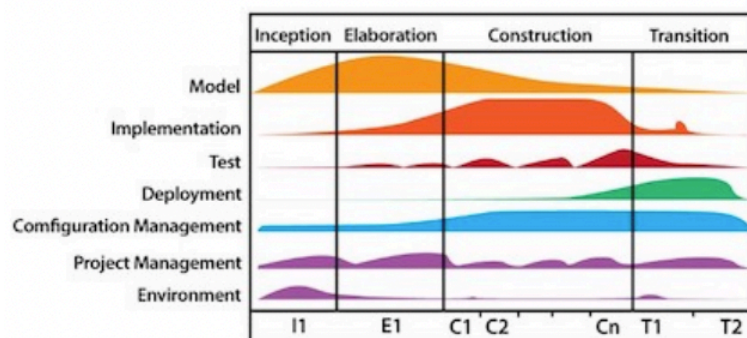


Figure 2-4 Ilustración: Proceso Unificado

Ventajas:

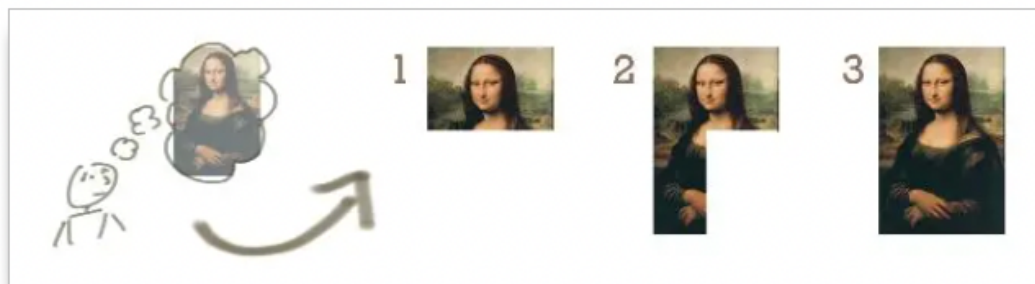
- Dirigido a casos de uso
- Centrado en la arquitectura
- Enfocado a **Riesgos Críticos** del proyecto

Desventajas:

- Mucho énfasis en **Documentación**
- **Costoso** de implementar
- Gestión y **Supervisión Alta**
- **Iterativo pero no siempre incremental** → **Limita adaptabilidad**

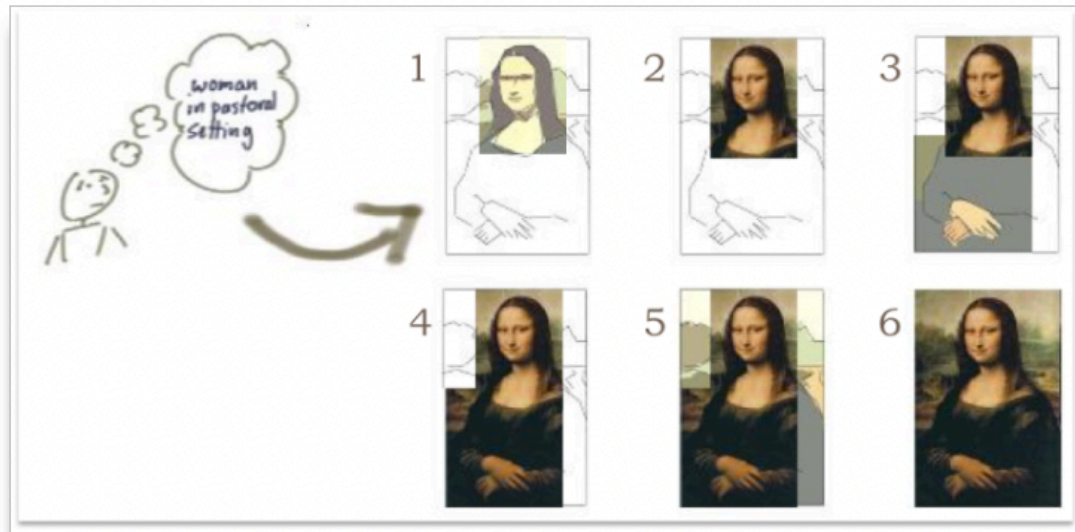
C) Incrementales:

- Con cada Entrega, tenemos acabada una **pieza** mas del sistema (Nuevas Funcionalidades), pero no esta acabado hasta entregar la ultima pieza
- **No Feedback** hasta terminar!



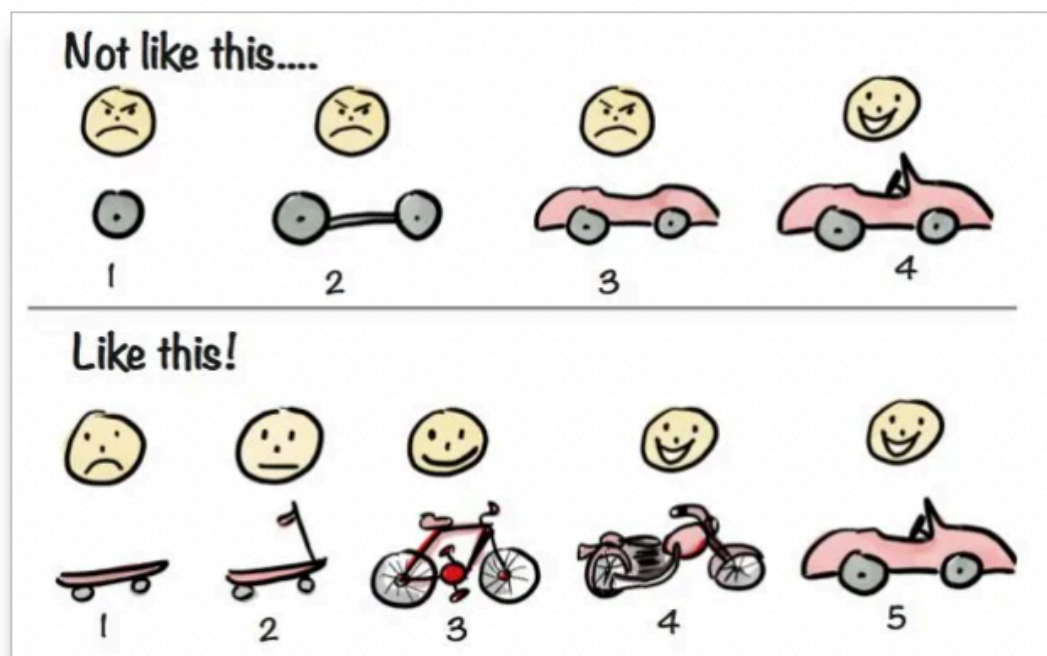
D) Iterativos Incrementales

- Con cada Entrega, añadimos **funcionalidades nuevas** (**INCREMENTAL**)
- Cada incremento incluye **mejoras** sobre funcionalidades ya existentes (**ITERATIVO**)



Ventajas:

- Adaptación, evitando cosas que no se deban hacer
- Entrega de Valor al Usuario, antes de acabar el desarrollo!!!



Desventajas:

- Pudiera ser que, en función del feedback recibido, tuviéramos que deshacer algo ya construido, con el consiguiente sobrecoste.
- Como queremos entregar incrementos de software frecuentemente, o automatizamos también nuestro proceso para poner el software funcionando a disposición de los usuarios, o vamos a incurrir en otro importante sobrecoste (no sólo en tiempo, sino también en errores provocados por las personas, que no somos tan buenos como las máquinas para hacer tareas repetitivas).

→ Modelo Agile

- Se **valora mas a Individuos** e Interacciones que procesos y herramientas.
- + **Software Funcional** - Documentación exhaustiva
- Colaboración con **Cientes**
- Flexibilidad



- Manifiesto Ágil:

1. *"Nuestra mayor prioridad es satisfacer al cliente a través de una entrega temprana y continua de software valioso para él"*
2. *"Abrazamos los cambios en los requerimientos, incluso si llegan tarde en el proceso".*

5. SCRUM

- Es útil para ambientes complejos o caóticos
- Promueve la **Flexibilidad** y **Colaboración** en el desarrollo de software
- La esencia de Scrum se reduce a los siguientes puntos:
 - El desarrollo de software se lleva a cabo en ciclos repetitivos llamados

"sprints", que representan períodos de tiempo específicos.

- Cada sprint agrega un incremento de funcionalidad al producto y genera un resultado "potencialmente entregable", es decir, un incremento de software funcional.
- El contenido del sprint se define en gran medida mediante una **negociación activa** con el representante del cliente, conocido como **"product owner"**.
- Al concluir cada sprint, se realizan dos reuniones esenciales antes de embarcarse en el siguiente ciclo:
- La reunión de **"review"**, que implica evaluar y presentar lo que se ha logrado durante el sprint, así como identificar lo que aún debe abordarse.
- La **"retrospectiva"**, una sesión reflexiva en la que el equipo analiza y discute cómo funcionó el proceso durante el sprint

anterior y cómo se pueden realizar mejoras.

ROLES:

1. Product Owner:

- Asume Responsabilidad total del proyecto: Exito o Fracaso
- Toma Decisiones claves
- Trabaja en estrecha colaboracion con los StakeHolders
- Identifica y prioriza las características esenciales que se incorporarán
- Garantiza que el equipo este construyendo el producto adecuado
- Y que el cliente este satisfecho

2. Scrum Master:

- Lider del proceso, es el Coach
- **Facilita y Guia** al equipo en la implementación de Scrum
- Promueve el ambiente propicio para la colaboración
- Asume responsabilidad de resolver problemas y el Scrum adecuado del equipo

3. Team:

- Es el equipo de desarrollo.
- Esta compuesto por individuos altamente colaborativos y habilidades diversas
- Son autónomos, 5-8 personas
- Planifican, diseñan, implementan y prueban características del producto

Actividades:

1. Sprint:

- Periodo de tiempo fijo y corto, se lleva a cabo el desarrollo
- Se crea un incremento del producto

2. Sprint Planning:

- Reunion al comienzo de cada sprint, donde el Equipo(+ScrumMaster) y Product Owner definen que elemento del Backlog se trabajaran y como durante el sprint

1. Objetivos del Sprint:

2. Sprint Backlog: To do / Doing / Done

3. Elaboran el plan

3. Daily Scrum:

- Reunion diaria y corta, donde el Equipo sincroniza sus Actividades
- Se revisa el progreso y discute obstáculos

4. Sprint Execution

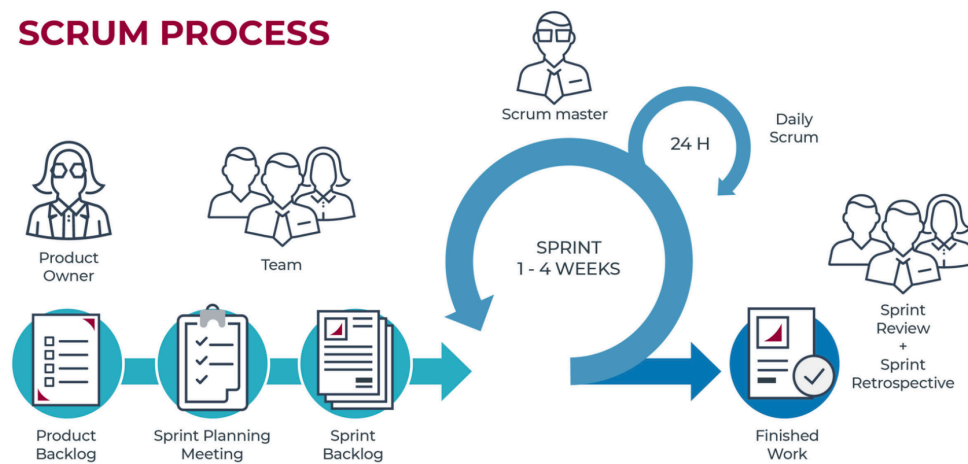
5. Sprint Review:

- Al final de cada sprint, se lleva a cabo una reunion
- Se revisa el incremento del producto, stakeholders proporcionan retroalimentación

6. Sprint Retrospective:

- Se reflexiona sobre el proceso, identificar que funciono bien y areas de mejora

SCRUM PROCESS



7. Relatos Usuarios:

Estructura:

- **Título:** Verbo y describe el objetivo claramente
- **Descripción:** Yo, como (type_usuario) necesito (tarea) para (objetivo)
- **Condiciones de Aceptación:** Pruebas para validar que se cumple la historia, entrega información adicional al desarrollador

Ejemplo 1

- Título: Recuperar contraseña.
- Descripción: Como usuario necesito recuperar mi contraseña para poder ingresar al sistema en caso de olvidar mi contraseña.
- Condiciones de aceptación:
- **Escenario 1:** Recuperación exitosa (happy path) Dado un usuario que navega a la página de inicio, Cuando el

usuario selecciona la opción “recuperar contraseña” e ingresa un email válido,

- Entonces el sistema debe enviar un enlace al email ingresado.
- Cuando el usuario entra al sistema a través del enlace,
- Entonces el sistema le permite al usuario establecer su nueva contraseña

8. Estimaciones: